

Coding Guidelines (Verilog-HDL)

Version 1.0

2002. 5. 25



*System Integration &
Intellectual Property Authoring Center*

<http://www.sipac.org>

Contents

- 1. FILE HEADER**
- 2. CODING FOR READABILITY**
- 3. CODING FOR PORTABILITY**
- 4. GUIDELINES FOR CLOCKS AND RESETS**
- 5. CODING FOR SYNTHESIS**

1. FILE HEADER

[1.1 필수] header는 모든 source file의 첫머리에 있어야 하며 아래의 항목이 포함되어 있어야 한다.

- **Title** : IP 이름
- **File** : Filename
- **Author** : name <E-mail>
- **Organization** : 기관이나 회사명
- **Created** : 만들어진 날짜
- **Last update** : 마지막 수정 날짜
- **Platform** : 사용 시스템
- **Simulator** : 검증 tool 명
- **Synthesizers** : Synthesis tool 명
- **Target** : 목표로 하는 Technology
- **Descriptions** : IP 서술, 주요 Key Feature
- **Revision Number** : 개정 번호
- **Version** : IP 버전
- **Date of change** : 개정날짜
- **Modifier** : 수정자 이름 <E-mail>
- **Description of change** : 개정 내용
- **Notice** : 사용자의 IP사용이 용이하기 위한 설계자 전달사항 (signal의 접미어 설계자 정의 등)

[Example]

//TITLE :	MISR
//FILE NAME :	misr_rtl.v
//AUTHER :	Kim Hyun Don(ihynun@yonsei.ac.kr)
//Organization :	Yonsei Univ comsys & RSoC Lab
//CREATED:	September 18, 2001
//LAST UPDATE :	September 11, 2001
//PLATFORM :	Win2000
//SIMULATOR :	ModelSim
//SYNTHESIZER :	Design Analyzer
//TARGET :	0.25u
//DISCRIPTION :	This module defines MISR using a single XOR gate
//REVISION NUMBER :	5213654
//VERSION NUMBER:	1.0
//DATE OF CHANGE :	December 25, 2001
//MODIFIER :	Seo Dae Sik (sds1403@yonsei.ac.kr)
//DESCRIPTION OF CHANGE :	IP's optimization
//NOTICE :	This Source Code is devised for the Digital Filter fault Test. The filter's internal bus and signal is tested.

2. CODING FOR READABILITY

[2.1 권장] 모든 signal, variable, port는 소문자를 사용해야 한다.

[Example] add, clk, rst

[2.2 권장] 모든 상수와 사용자 정의 형식의 이름은 대문자를 사용해야 한다.

[Example] BITS, DEL

[2.3 필수] 대소문자로 구분된 이름을 사용해서는 안된다.

[Example] OUT, out

[2.4 권장] 다음의 signal 이름 규약에 따라야 한다.

- *_r: register 출력
- *_a : 비동기 signal
- *_pn : n 번째 phase로 사용되는 signal (예, enable_p2)
- *_nxt : 같은 register로 입력되기 전의 data
- *_z: tristate 내부 signal

[2.5 권장] signal 이름이 12글자를 초과해서는 안된다.

[Example] signal_flow_diagram



[2.6 필수] clock signal 이름은 “clk” 또는 “clk”를 접두사로 사용해야 한다.

[Example] clk1, clk_interface

[2.7 필수] 같은 source에서 나온 모든 clock signal은 같은 이름을 사용해야 한다.

[2.8 권장] active low signal 이름은 underscore와 함께 소문자를 사용해야 한다.

[Example] rst_n, preset_n

[2.9 필수] reset signal은 “rst” 또는 “rst”를 접두사로 사용해야 한다.

[Example] rst_n

[2.10 필수] Verilog-HDL이나 VHDL의 예약어를 naming에 사용하지 않아야 한다. 다음은 Verilog-HDL 과 VHDL의 예약어를 보여준다.

Verilog-HDL 예약어

always, and, assign, begin, buf, bufif0, bufif1, case, casex, casez, cmos, deassign, default, defparam, disable, edge, else, end, endattribute, endcase, endmodule, endfunction, endprimitive, endspecify, endtable, endtask, event, for, force, forever, fork, function, highz0, highz1, if, ifnone, initial, inout, input, integer, join, large, macromodule, medium, module, nand, negedge, nmos, nor, not, notif0, notif1, or, output, parameter, pmos, posedge, primitive, pull0, pull1, pulldown, pullup, rcmos, real, realtime, reg, release, repeat, rmos, rpmos, rtran, tranif0, tranif1, scaled, signed, small, specify, specparam, strength, strong0, strong1, supply0, supply1, table, task, time, tran, tranif0, tranif1, tri, tri0, tri1, triand, trior, trireg, unsigned, vectored, wait, wand, weak0, weak1, while, wire, wor, xor, xnor

VHDL 예약어 (not exist in Verilog-HDL)

abs, access, after, alias, all, architecture, array, assert, attribute, block, body, buffer, bus, component, configuration, constant, disconnect, downto, elsif, entity, exit, file, generate, generic, group, guarded, impure, in, inertial, is, label, library, linkage, literal, loop, map, mod, on, open, others, out, package, port, postpone, procedure, process, pure, range, record, register, reject, rem, report, return, rol, ror, select, severity, signal, shared, sla, sll, sra, srl, subtype, then, to, transport, type, unaffected, units, until, use, variable, when, with

[2.11 필수] port 선언 순서는 다음을 따라야 한다.

● **INPUTS**

1. Clocks
2. Resets
3. Enables
4. Other control signals
5. Data and address lines

● **OUTPUTS**

1. Clocks
2. Resets
3. Enables
4. Other control signals
5. Data

[Example] module signal_multiply (clk, rst, multiply_en, a, b, product, valid) ;

[2.12 권장] port 이름은 12글자를 초과해서는 안된다.

[2.13 권장] 입력과 출력 port 사이에는 blank line을 두어야 한다.

[Example]

```
module adder (a, b, temp, out) ;
```

```
input a, b ;
```

```
input temp ;
```

```
output out ;
```

```
.....
```

Blank line

[2.14 권장] function의 이름은 12글자를 초과해서는 안된다.

[Example] function address_count ;

function addr_cnt ;



[2.15 권장] 같은 코드를 반복하여 사용하는 경우, 가능하면 **function**을 사용하고 그 기능에 대한 설명을 해야 한다.

[Example]

Verilog

// This function converts the incoming address to the corresponding relative address.

```
function [BUS_WIDTH-1:0] conv_add;  
    input input_add, offset;  
    integer input_add, offset;  
begin
```

// Function body goes here

```
end  
end function    // conv_add
```

[2.16 권장] loop 와 array를 가급적 사용하되, 가능하면 loop 보다는 array를 사용하는 것을 권장한다.

[2.17 권장] 각 instance의 이름은 “U#_<name>”으로 해야 한다.
(# : instance 번호)

[Example] flipflop U0_flop flop(clk, rst_n, din, qout) ;

[2.18 필수] ASIC library에 존재하는 같은 instance cell 이름은 사용하지 않아야 한다.

[2.19 권장] 각 HDL 문장은 분리된 라인을 사용해야 한다.

[Example]

```
check_out <= #`DEL check_out + 1 ;  
four_count <= #`DEL four_count +1 ;  
count <= #`DEL count + 1 ;
```

[2.20 권장] line당 글자수가 132자를 넘지 않아야 한다.

[2.21 권장] 계속되는 code line과 loop에는 들여쓰기를 사용해야 한다.

[Example]

```
for (i = 0; i < 32; i = i + 2)  
    count[i] = 0;
```

[2.22 권장] 들여쓰기는 1~4개의 space를 사용하되 그 수가 일정해야 한다.

[2.23 권장] 모든 process, function, type과 subtype의 선언에 대해 comment가 있어야 한다.

[Example]

```
// parity calculation function  
function calc_parity;  
//Type  
integer cycle_count; // cycle count variable  
integer cycle_limit; // number of cycles to simulate
```


[2.24 권장] port, signal, variable은 line당 한 개씩 선언해야 하며, 같은 line에 이에 대한 comment가 있어야 한다.

[Example]

```
module adder_8bit(clk, rst_n, add_en, a, b, cin, out, cout);  
input clk; // clock  
input rst_n; // active low, synchronous reset  
input add_en; // synchronous add enable control  
input [BITS-1:0] a; // 8bit a input  
input [BITS-1:0] b; // 8bit b input  
input cin; // carry in
```

[2.25 권장] 2개 이상의 산술연산의 경우, priority가 명백하더라도 readability를 위해 괄호를 사용하라.

[2.26 필수] 비교 연산자의 비교 대상은 bit width가 일치 해야 한다.

[2.27 권장] 5bit 이상의 상수를 지정할 때 항상 bit width(S`b, S`h, S`d, S`o)를 명기해야 한다.

[Example]

```
wire[7:0] s8;  
wire[15:0] s16;  
wire s1;  
  
assign s16 = s8;  
assign s16 = {s8, s1};  
assign s8 = s16;
```



```
wire[7:0] s8;  
wire[15:0] s16;  
wire s1;  
  
assign s16 = {8'h00, s8};  
assign s16 = {7'h00, s8, s1};  
assign s8 = s16[7:0];
```

[2.28 필수] 다중 비트 **signal** 이나 **port**일 경우 “(0 : x)” 보다 “(x : 0)”를 사용해야 한다.

[Example]

```
input [BITS-1:0] a;
```

[2.29 필수] **port**와 **generic**에 대한 연결은 위치에 의한 것보다도 이름에 의한 것으로 하여야 한다.

[Example]

```
Adder_8bit U0_Add(a(a[ BITS-1:0]),,b(b[ BITS-1:0]),,cin(carry_in));
```

[2.30 권장] 모든 **identifier**는 소문자를 사용하고 **definition**은 대문자를 사용한다.

[2.31권장] **Module** 이름은 {**IP name** 또는 약어} + { **_name**}로 하며, **source file name**과 **module name**은 일치 시킨다.

[Example]

▪ **source file** 이름 : **arm_test.v**

```
module arm_test (.....)
.....
end module
```

[2.32 권장] **module** 이름은 12자를 넘어서는 안된다.

[2.33 권장] `always` 문에서, 각각의 `reg variable`의 사용에 `comment`를 해야 한다.

[2.34 권장] `always`문의 라인은 200개를 넘어서는 안된다.

3. CODING FOR PORTABILITY

[3.1 권장] hard-coded 숫자는 사용하지 않아야 한다. (1과 0은 제외)

[Example]

```
`define BUS_SIZE 8
wire [ `BUS_SIZE-1:0] in_bus;
reg [ `BUS_SIZE-1:0] out_bus;
```

[3.2 필수] technology independent library를 사용해야 한다.

[3.3 권장] 같은 설계에서 active low logic과 active high logic을 섞어 쓰지 않아야 하며, active high logic을 권장한다.

[3.4 권장] core 상호간의 연결은 최소의 wire를 사용하여 설계해야 한다.

[3.5 권장] 일반적으로 내부의 tri-state signal의 사용을 피해야 한다. 하지만 내부 모니터를 위한 tri-state는 권장한다.

[3.6 권장] generic_spram 이나 generic_dpam과 같은 동기식 single-port 또는 dual-port 메모리의 사용을 권장한다.

[3.7 필수] 나누기 연산을 사용하지 않아야 한다.

[3.8 권장] **constant**를 정의할 때 **definition(define)** 또는 **parameter**를 이용해야 한다.

[3.9 권장] **constant**는 분리된 파일에 정의하고 이 파일을 **include** 하여 사용하여야 한다.

[3.10 권장] **constant**가 정의된 **include** 파일명은 **p_<design name>.v**로 해야한다.

[3.11 권장] 계층구조에서 각 계층에서만 사용되는 **parameter** 이름은, 계층 ID를 사용하여 전체에 사용되는 **parameter** 이름과 구분하여야 한다.

[Example]

전체에 사용되는 **parameter (common include file)**

parameter MAX_WIDTHA = 32 ;

parameter MAX_WIDTHB = 16 ;

한 계층에서만 사용되는 **parameter (include file in each layer)**

parameter MAX_LOCAL_WIDTHA = 32 ;

parameter MIN_LOCAL_WIDTHA = 8 ;

[3.12 권장] 효율적인 설계를 위하여, **always** 또는 **function+assign** 구문을 사용하라.

[참고]

	function+assign	always
Merit	▪ latch를 생성하지 않는다 ▪ combinational 과 sequential 회로의 구분이 명확하다	▪ simulation 속도가 빠르다 ▪ 기술이 간단하다
Demerit	▪ bit width의 에러가 있을 수 있다 ▪ 모든 argument가 선언되지 않더라도 syntax error가 없을 수 있다 ▪ 단지 한 개의 출력만 가능하다	▪ latch를 생성할 수 있다 ▪ sensitivity list를 완전히 기술하지 않으면, RTL과 gate simulation의 결과가 다를 수 있다 ▪ 다중 출력을 사용하면, 비교적 큰 회로가 생성된다

[3.13 권장] 단일 bit 연산일 경우라도, 논리 연산자 보다는 **bit-wise** 연산자를 사용해야 한다.

[3.14 필수] 논리 연산을 다중 bit 연산에 사용해서는 안된다.

[3.15 권장] vector를 if 문이나 conditional operator (?:)의 conditional expression으로 사용해서는 안된다.

[Example]

<pre>reg [3:0] AVECTOR ; if (AVECTOR)</pre>	X	➔	<pre>reg [3:0] AVECTOR ; if (AVECTOR == 4'h0)</pre>
---	---	---	---

[3.16 필수] 1차원의 array의 경우, [MSB:LSB]를 사용해야 한다.

[3.17 권장] array의 LSB는 가능하다면, 0으로 하는 것이 좋다.

[3.18 권장] 하나의 always문의 출력은 5개 이거나 그보다 적어야 한다.

[3.19 권장] 하나의 always문에서 다수의 출력을 정의해서는 안된다.

[Example]

```
always @ (a or ain or bin or c or s_tmp or z)
begin
  if (a == 1'b1)
    s_tmp <= ain + bin ;
  else
    s_tmp <= ain - bin ;
  c <= s_tmp[15] ;
  if (s_tmp[14:0] == 15b`0 )
    z <= 1'b1 ;
  else
    z <= 1'b0 ;
  if (c == 1 && Z == 1'b0 )
    cout <= 1'b1 ;
  else
    cout <= 1'b0 ;
  sout <= s_tmp[14:0] ;
end
```



```
always @ (a or ain or bin) begin
  if (a == 1'b1)
    s_tmp <= ain + bin ;
  else
    s_tmp <= ain - bin ;
end
always @ (s_tmp) begin
  if (s_tmp[14:0] == 15b`0 )
    z <= 1'b1 ;
  else
    z <= 1'b0 ;
end
assign c = s_tmp[15] ;
always @ (c or z ) begin
  if (c == 1 && Z == 1'b0 )
    cout <= 1'b1 ;
  else
    cout <= 1'b0 ;
end
assign sout <= s_tmp[14:0] ;
```

4. GUIDELINES FOR CLOCKS AND RESETS

[4.1 권장] 전체 설계를 통해 단일 clock phase flip-flop (positive edge 또는 negative edge)을 사용해야 한다.

[4.2 필수] positive-edge 와 negative-edge triggered flip-flop이 모두 사용되었다면 worst case의 duty cycle에 대한 timing 분석과 synthesis를 위한 모델링이 되어 있어야 한다.

[4.3 필수] positive-edge 와 negative-edge triggered flip-flop이 모두 사용되었다면 duty cycle 값이 서술되어 있어야 한다.

[4.4 권장] positive-edge 와 negative-edge triggered flip-flop이 모두 사용되었다면 각 phase의 설계를 다른 모듈로 분리해야 한다.

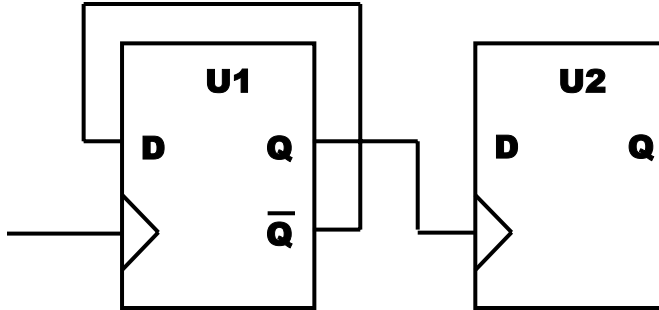
[4.5 권장] clock 생성 회로는 분리된 module 또는 entity로 설계해야 한다.

[4.6 필수] clock buffer는 사용하지 말아야 한다. synthesis후에 physical design 단계에서 삽입한다.

[4.7 권장] gated clock은 사용하지 않아야 한다.

[4.8 권장] 설계 내에서 생성되는 **clock**을 사용하지 말아야 한다.

[Example]



Bad Example : 내부 생성 clock

[4.9 필수] **gated clock** 이나 설계 내에서 생성되는 **clock, reset**을 사용해야 된다면 이 회로들은 **top level**에서 다른 회로들과 구분되어 있어야 한다.

[4.10 권장] 설계 내에서 생성된 **conditional reset**은 없어야 한다. **macro** 전체는 한번에 **reset** 되어야 한다.

[4.11 필수] **conditional reset**이 필요하다면 **reset line**을 위한 분리된 **logic**을 만들고 이 **logic**을 다른 모듈과 분리시켜야 한다.

[4.12 권장] **clock** 또는 **reset**을 **data** 또는 **enable**로 사용하지 말고, **data**를 **clock** 또는 **reset**으로 사용해선 안된다.

[4.13 권장] 레지스터의 초기 **reset**을 위하여 **asynchronous reset**을 사용하라.

[4.14 권장] multiple clock을 사용할 때는 각 clock은 분리된 블록을 가져야 한다.

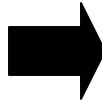
[4.15 권장] 논리합성시, clock tree optimization을 위한 layout에서 얻은 clock skew 값을 예측하여 제공해야 한다.

[4.16 필수] latch와 asynchronous set/reset을 함께 사용해서는 안된다.

[Example]

```
//Latch with asynchronous reset
module (q, g, data, reset_x)
  output q ;
  input q, data, reset_x ;
  reg q ;

  always @ (g or data or reset_x)
    if (!reset_x)
      q <= 0 ;
    else if (g)
      q <= data ;
endmodule
```



```
//Latch
module (q, g, data)
  output q ;
  input q, data ;
  reg q ;

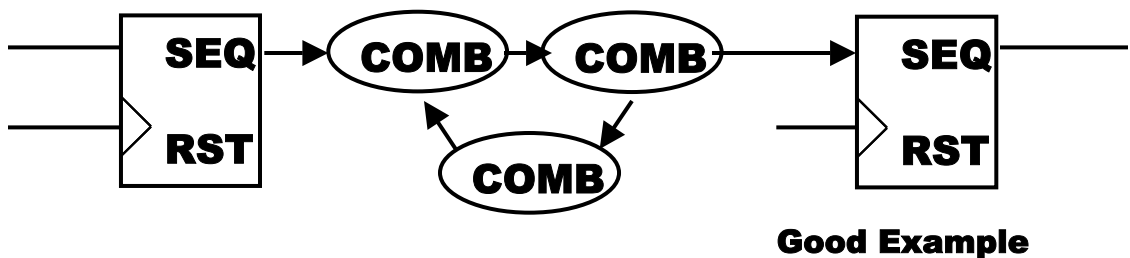
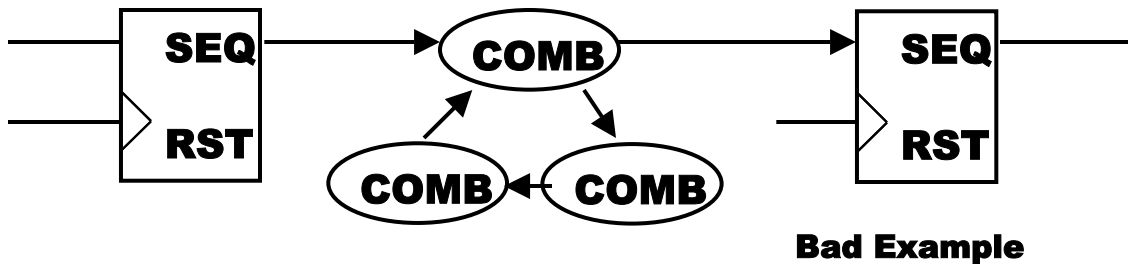
  always @ (g or data)
    if (g)
      Q <= data ;
endmodule
```

5. CODING FOR SYNTHESIS

[5.1 필수] RTL에서 latch가 추출되지 않게 해야 한다.

[5.2 필수] combinational feedback은 사용하지 마라. 즉 combinational process loop는 없어야 한다.

[Example]



[5.3 필수] 각 process의 sensitivity list에 꼭 필요한 signal만 있어야 한다

[5.4 권장] “if A=case_1 ~, elsif A=case_2 then ~,, else ~ end if” 문장보다 “case” 문장사용을 권한다.

[Example]

```
EX_PROC: process ( sel, a, b, c, d )
begin
  case sel is
    when 00 => outc <= a;
    when 01 => outc <= b;
    when 10 => outc <= c;
    when others => outc <= d;
  end case;
end process EX_PROC;
```

[5.5 권장] FSM logic과 non-FSM logic은 각 다른 모듈로 구분해야 한다.

[Example]

```
module fsm (clock, rst, x, z);
  input clock, rst, x;
  output z;
  reg [1:0] current_state;
  reg [1:0] next_state;
  reg z;
  parameter [1:0]
  STATE_0 = 0,
  STATE_1 = 1,
  STATE_2 = 2,
  STATE_3 = 3;

  // combinational process calculates next state

  always @ ( current_state or x )
  case ( current_state ) // synopsys parallel_case full_case
    STATE_0 : begin
      if (x) begin
        next_state = STATE_1;
        z = 1'b0;
      end else begin
        next_state = STATE_0;
        z = 1'b0;
      end
    end
    STATE_1 : begin
      if (x)
        begin
          next_state = STATE_2;
          z = 1'b0;
        end
    end
  endcase
```

```

else
  begin
    next_state = STATE_1;
    z = 1'b0;
  end
end
STATE_2 : begin
  if (x)
    begin
      next_state = STATE_3;
      z = 1'b0;
    end
  else
    begin
      next_state = STATE_2
      z = 1'b0;
    end
end

STATE_3 : begin
  if (x)
    begin
      next_state = STATE_0;
      z = 1'b1;
    end

```

```

else
  begin
    next_state = STATE_3;
    z = 1'b0;
  end
end

default : begin
  next_state = STATE_0;
  z = 1'b0;
end
endcase

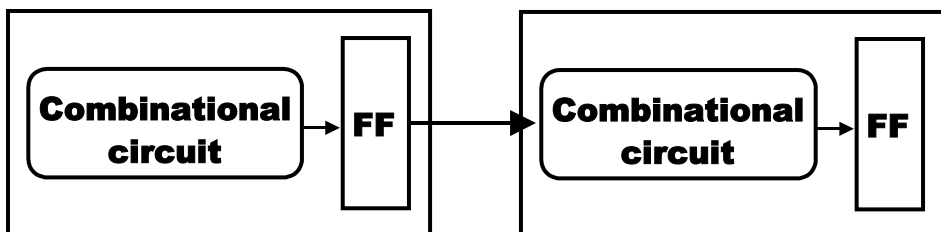
always @ ( posedge clock or negedge rst_na
)
  begin
    if ( !rst_na )
      current_state = STATE_0;
    else
      current_state = next_state;
    end
  end
endmodule

```

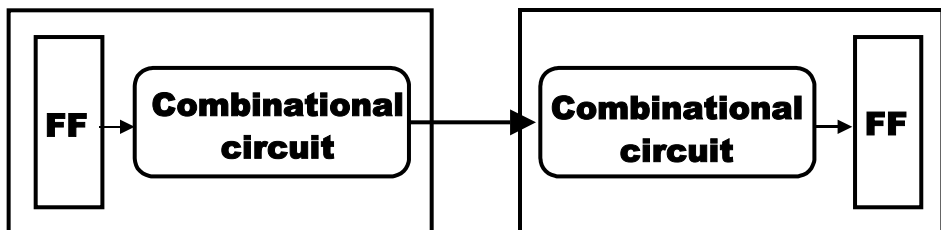
[5.6 권장] state machine의 HDL은 combinational logic과 sequential logic으로 분리하여 기술해야 한다.

[5.7 필수] 계층적 설계의 각 block의 모든 출력신호는 register로부터 직접 나와야 한다. Timing이 critical하지 않을 경우, register에 입력에 combinational logic 출력을 사용하라.

[Example]



권장하는 기본 구조



두개의 블록 path를 갖는 경우

[5.8 권장] critical path logic은 non-critical path logic과 분리된 모듈로 있어야 한다.

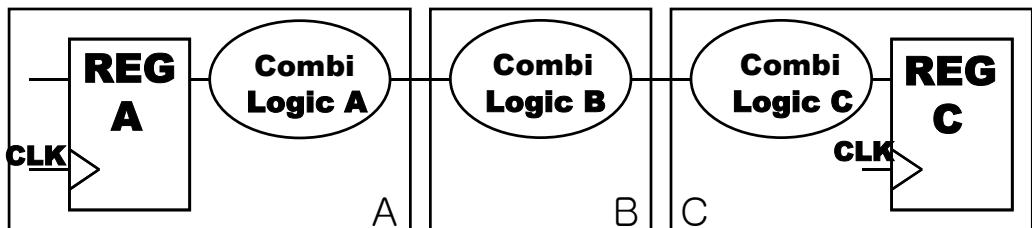
[5.9 권장] asynchronous logic은 피해야 한다.

[5.10 권장] asynchronous logic이 사용되었다면 synchronous logic과 module을 분리하고 structural 형식보다 behavioral로 모델링 되어야 한다.

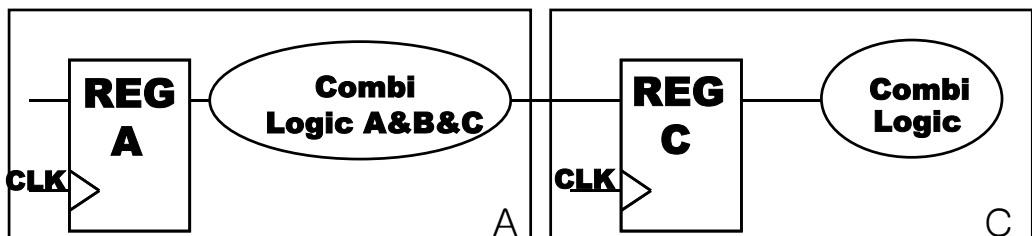
[5.11 필수] Data path 블록과 control 블록은 분리시켜라.

[5.12 권장] 관련된 combinational logic은 같은 모듈에 함께 있어야 한다.

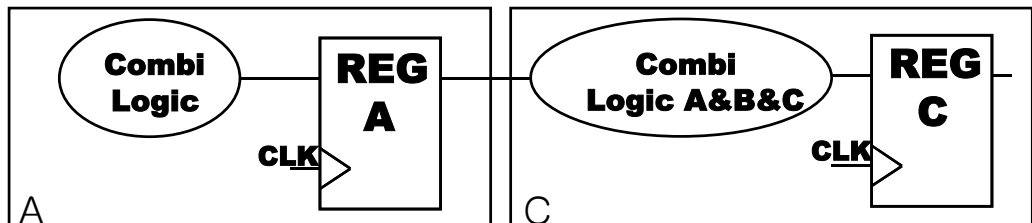
[Example]



Bad Example



Batter Example



Best Example

[5.13 권장] arithmetic equation을 사용시 synthesis에서 자원 공유를 위해 같은 hierarchy level의 모듈로 분리해야 한다.

[5.14 권장] multicycle path는 피해야 한다.

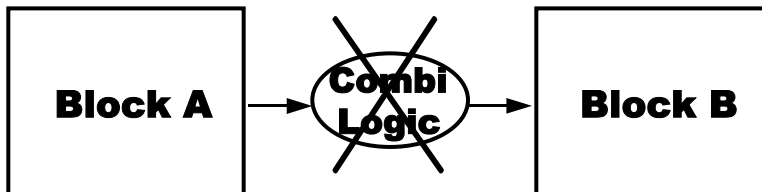
[5.15 권장] multicycle path가 있어야 한다면 point-to-point exceptions가 단일 모듈안에 있어야 하고 comment가 있어야 한다.

[5.16 권장] false path는 피해야 한다.

[5.17 권장] false path가 있어야 한다면 이를 잘 설명해야 한다.

[5.18 권장] 상위 구조는 하위 블록으로 구성되어야 하며 어떠한 combinational logic도 포함해서는 안된다.

[Example]



[5.19 권장] top level에서는 어떠한 로직도 추가하지 말고 단지 각 블록의 연결정보만을 기술해야 한다.

[5.20 권장] 메모리 사용시 address, data register 와 write enable logic을 분리된 모듈로 만들어야 한다.

[5.21 필수] delay element를 사용하지 말아야 한다. 이는 P&R에서 수행된다

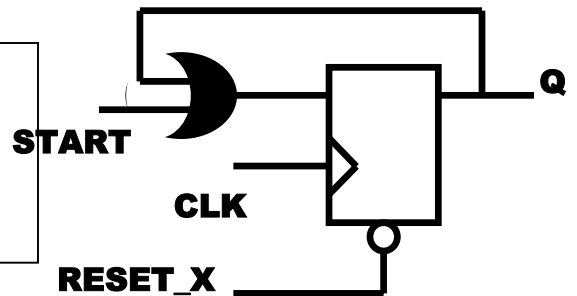
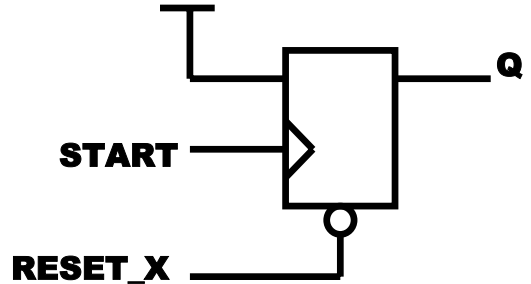
[5.22 권장] 고정된 입력 값을 갖는 Flip-Flop을 기술하지 않아야 한다.

[Example]

```
always @ (posedge start or negedge reset_x) begin
  if (!reset_x)
    q <= 1'b0 ;
  else
    q <= 1'b1 ;
end
```



```
always @ (posedge clk or negedge reset_x) begin
  if (!reset_x)
    q <= 1'b0 ;
  else if (start)
    q <= 1'b1 ;
end
```



[5.23 권장] latch를 기술하는 구문과 다른 combinational 회로는 정확히 구분하여야 하고, 분리된 hierarchy 구조를 사용하여야 한다.

[5.24 권장] critical timing을 갖거나 변화가 많은 signal은 conditional branch의 초기에 설정해 주어야 한다.

[Example]

```
module DECODE(y, a, b, c, s1, s2);
  output y;
  input a, b, c, d;
  reg f;
  always @ (a or b or c or s1 or s2) begin
    if (s1 == 1'b1)
      y = a ;
    else if (s2 == 1'b1)
      y = b;
    else
      y = c;
  end
endmodule
```


[5.25 권장] if 문은 priority logic을 생성하기 때문에 nesting value는 적을 수록 좋다.

[5.26 권장] if 문에서 nest수는 최대 10 이상이 되어서는 안된다.

[5.27 필수] case 문을 사용할 경우, 항상 default 문을 함께 사용해야 한다.

[5.28 필수] case 문은 input과 output(input : 16, output : 8)을 포함하여 24bit를 넘어서는 안되며, case의 item은 100개를 넘어서는 안된다.

[5.29 권장] 공통된 값을 branch condition으로 사용할 경우, if 문을 사용하여 비교하고, 이를 case 문으로 다시 비교하여 구문을 간략화 하는 것이 좋다.

[Example]

```
case (a)
  32`b0000000010000000001111011011100110 : dout <= 5`b00000;
  32`b0000000010000000001111011111110110 : dout <= 5`b00010;
  32`b0000000010000000001101011011100110 : dout <= 5`b00011;
  32`b0000000010000000001111011011110110 : dout <= 5`b00100;
  32`b0000000100000000001110011011100110 : dout <= 5`b00101;
  32`b0000000100000000001111011011100110 : dout <= 5`b00110;
  .....
```



```
if (A[25:24] == 2`b01) begin
  case ( A[18:0] )
    .....
  else if (A[25:24] == 2`b10) begin
    case ( A[18:0] )
      .....
```

[5.30 권장] don't care condition은 default 문으로 'X'를 사용하여 정의하여야 한다. default 문으로 고정된 값을 지정하면, 회로의 크기가 커질 수 있다.

[5.31 필수] case 문에서 default로 don't care로 지정된 signal은 if 문에서 사용되어서는 안된다.

[Example]

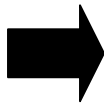
```
always @ ( a )
  case ( y )
    ...
    default : y = 4`bxxxx ;
  endcase
always @ (posedge clk or negedge rst_x )
  if (lrst_x)
    dout <= 1`b0;
  else if (y == 4`b0000)
    dout <= #DLY 1`b1;
  else
    dout <= #DLY 1`b0;
```



[5.32권장] if 문과 case 문이 함께 사용될 경우, 큰 table안에 적은 조건이 있는 것보다 적은 table안에 많은 조건이 있는 것이 좋다.

[Example]

```
case (X) is
  .... : if (....) else
  .... : if (....) else
  .... : if (....) else
  .... : if (....) else
  .... : if (....) else
```



```
if (....)
  case (X)
    .... : ....
    .... : ....
    .... : ....
  .
  .
  .
else
  case (X)
    .... : ....
    .... : ....
    .... : ....
```

[5.33 필수] for 문의 초기값과 조건은 상수이어야 하며, loop variable 안에서 그 값이 바뀌어서는 안된다.

[Example]

```
integer I;  
  
always @ (INA) begin : loop  
    F = INA[0] ^ INA[1] ;  
    for ( I = 2; I <= 7; I = I + 1 )  
        F = F ^ INA[ I ];  
end
```

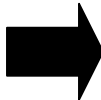


[5.34 필수] for 문에서 loop variable이나 loop constant 이외에는 산술연산을 수행하지 않아야 한다.

[Example]

***Description example (a) :**

```
for ( i = 0; i <= 7; i = i + 1 ) begin  
    if (i >= a - 1)  
        S[ i ] = 1'b1;  
    else  
        S[ i ] = 1'b0;  
end
```



***Description example (b) :**

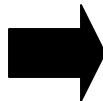
```
TMP = a - 1;  
for ( i = 0; i <= 7; i = i + 1 ) begin  
    if (i >= TMP)  
        S[ i ] = 1'b1;  
    else  
        S[ i ] = 1'b0;  
end
```

[5.35 권장] 가능하다면, for 문 보다는 case 문을 사용하는 것이 좋다.

[Example]

***Description example (a) :**

```
TMP = a - 1;  
for ( i = 0; i <= 7; i = i + 1 ) begin  
    if (i >= tmp)  
        S[ i ] = 1'b1;  
    else  
        S[ i ] = 1'b0;  
end
```



***Description example (b) :**

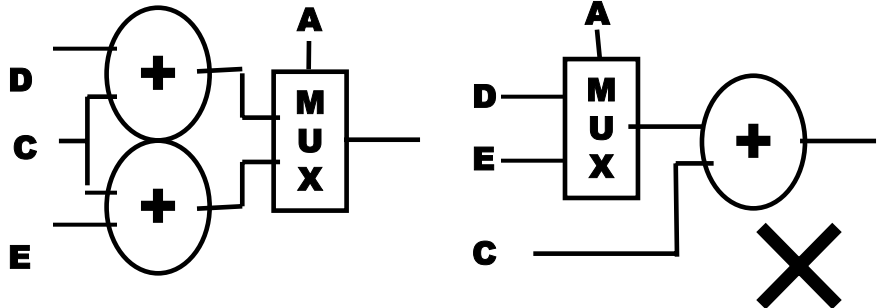
```
assign TMP = a - 1;  
always @ (tmp) begin  
    case ( tmp )  
        3'b000 : s = 8b`11111111;  
        3'b001 : s = 8b`11111110;  
        3'b010 : s = 8b`11111100;  
        3'b011 : s = 8b`11111000;  
        3'b100 : s = 8b`11110000;  
        3'b101 : s = 8b`11100000;  
        3'b110 : s = 8b`11000000;  
        3'b111 : s = 8b`10000000;  
        default : s = 8b`xxxxxxxx;  
    endcase  
end
```

[5.36 필수] integer 또는 signal register 나 wire에 음의 값을 지정하지 않아야 한다.

[5.37 권장] 속도가 중요한 회로에는 resource 를 공유하지 말아야 한다.

[Example]

```
if (A == 1'b1)
  B <= C + D ;
else
  B <= C + E ;
```



[5.38 필수] always 문에 sensitivity list를 완전히 기술해야 한다.

[Example]

```
always @(alu_side or shift_left or shift_right or in_flags)
begin
  ....function body here....
end
```

[5.39 필수] nonblocking assignment는 synthesis를 위해 always @ (posedge clk) 문을 사용해야 한다.

[Example]

```
always @ ( posedge clk )
begin
  b <= a;
  a <= b;
end
```

[5.40 필수] state machine의 default state를 default 문으로 정의해야 한다.

[5.41 필수] “assign #x a = b;” 또는 “assign a = #x b;” (x는 delay의 단위시간 수)와 같은 구문은 사용하지 말아야 한다.

[5.42 권장] signal이나 variable에 초기값을 할당하는 구문을 사용하지 말아야 한다.

[Example]

wire b=1'b0; (X)

[5.43 권장] non-blocking assignment(<=)는 function 문에서 사용하지 않아야 한다.

[5.44 권장] function 내의 모든 argument는 function 입력으로 정의 되어야 한다.

[Example]

```
module MUX(y, a, b, c, d, s) ;
    output y ;
    input a, b, c, d ;
    input[1:0] s ;

    function FUNC_MUX ;
        input a, b, c, d ;
        input[1:0] s ;
        begin
            case (s)
                2'b00 : FUNC_MUX = a ;
                2'b01 : FUNC_MUX = b ;
                2'b10 : FUNC_MUX = c ;
                2'b11 : FUNC_MUX = d ;
            endcase
        end
    endfunction

    assign y = FUNC_MUX(a, b, c, d, s) ;

endmodule
```

[5.45 권장] combinational 회로의 always 문에서는 blocking assignment (=)를 사용해야 한다.

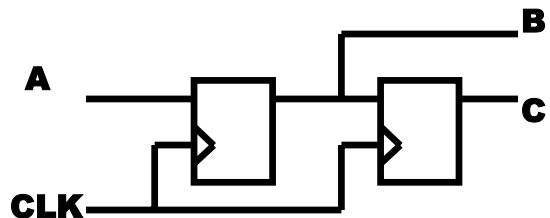
[5.46 필수] always 문에서 초기값 설정을 위해 non-blocking assignment를 사용하지 않아야 한다.

[5.47 필수] sequential 회로의 always 문에서는 초기값을 설정하지 않아야 한다.

[5.48 필수] Flip-Flop 회로를 생성하기 위해서, non-blocking assignment (<=)를 사용해야 한다.

[Example]

```
always @(posedge clk) begin
    b <= a ;
    c <= b ;
end
```



[5.49 권장] argument의 bit width와 function 입력선언의 bit width를 일치시켜야 한다.

[5.50 필수] function 문은 return 값을 assign하는 것으로 마쳐야 한다.

[Example]

```
module DEC2TO4 (ain, en, out, f) ;
  input [1:0] ain ;
  input      en ;
  output [3:0] out ;
  output f ;

  function [3:0] DEC ;
    input [1:0] ain ;
    begin
      if ( en )
        case ( ain )
          2`h0 : DEC = 4`b0001 ;
          2`h1 : DEC = 4`b0010 ;
          2`h2 : DEC = 4`b0100 ;
          2`h3 : DEC = 4`b1000 ;
          default : DEC = 4`bxxxx ;
        endcase
      else
        DEC = 4`b0000 ;
      end
    endfunction
  assign {f, out} = DEC ( ain ) ;
endmodule
```

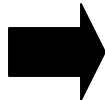
bit width is not matched

[5.51 필수] if 문에서 여러 개의 expression이 사용될 경우 block statement로 구분 되어야 한다.

[Example]

Example description

```
always @ (a or b or c or s)
begin
  if (s == 1`b0)
    y = 1`b0;
  else if (a == 1`b1)
    y = 1`b1;
  if (b == 1`b1)
    z = 1`b0;
  else begin
    y = c;
    z = 1`b1;
  end
end
end
```



Simulator interpretation

```
always @ (a or b or c or s)
begin
  if (s == 1`b0)
    y = 1`b0;
  else if (a == 1`b1)
    y = 1`b1;

  if (b == 1`b1)
    z = 1`b0;
  else begin
    y = c;
    z = 1`b1;
  end
end
end
```

[5.52 필수] function 문 안에서, global signal assignment는 사용해서는 안된다.

[Example]

```
module DEC2TO4 (ain, en, out) ;  
  input [1:0] ain ;  
  input      en ;  
  output [3:0] out ;  
  reg [3:0] out ;  
  wire f ;
```

```
  function [3:0] DEC ;
```

```
    inout en ;
```

```
    input [1:0] ain ;
```

```
    begin
```

```
      if ( en )
```

```
        case ( ain )
```

```
          2`h0 : out = 4`b0001 ;
```

```
          2`h1 : out = 4`b0010 ;
```

```
          2`h2 : out = 4`b0100 ;
```

```
          2`h3 : out = 4`b1000 ;
```

```
          default : out = 4`bxxxx ;
```

```
        endcase
```

```
      else
```

```
        out = 4`b0000 ;
```

```
      end
```

```
    endfunction
```

```
    assign f = DEC ( en, ain ) ;
```

```
  endmodule
```

Assign to global signal



[5.53 권장] 가능하면 복잡한 casex문을 사용하지 않는 것이 좋다.

[5.54 권장] casex 문에서 규칙적으로 don't care 값(x)을 기술하는 것이 좋다.

[Example]

```
casex ( a )  
  8`bxxxxxxxx0 : y = 3`b111;  
  8`bxxxxxxxx01 : y = 3`b110;  
  8`bxxxxxx011 : y = 3`b101;  
  8`bxxxxx0111 : y = 3`b100;  
  8`bxxx01111 : y = 3`b011;  
  8`bxx011111 : y = 3`b010;  
  8`bx0111111 : y = 3`b001;  
  default :      y = 3`b000;  
endcase
```


[5.55 권장] case selection 표현 내에 논리연산이나 산술 연산을 기술해서는 안된다.